EC4307 WEB APPLICATION DEVELOPMENT

BoardEase

SMART BOARDING FINDER

Mini Project Report

A MERN-stack platform for discovering and managing student boarding spaces.

STUDENT INFORMATION	DETAILS
Student name	[Enter your full name]
Index number	[Enter your index number]
Course / Module	EC4307 Web Application Development
Submission date	31 October 2026

ACTION REQUIRED Replace student details, add actual test evidence and screenshots, then rename the report with your index number.

1. Project Overview

BoardEase is a web-based Smart Boarding Finder created to help students search for suitable accommodation and help boarding owners publish and manage property listings. The platform presents essential information such as location, monthly rent, room type, facilities and availability in a single interface.

1.1 Problem Statement

Students often rely on scattered advertisements and personal contacts when looking for boarding accommodation. Details may be incomplete or hard to compare. Owners also need an easier way to add, update and remove available rooms. BoardEase addresses these problems with searchable property listings and a simple management dashboard.

1.2 Objectives

- Create a student-friendly, responsive browsing experience.
- Implement persistent Create, Read, Update and Delete (CRUD) operations for boarding listings.
- Allow users to filter and sort properties by practical criteria.
- Provide owner account access and basic listing ownership protection.
- Display clear validation and error feedback.

1.3 Scope

Students and visitors can browse, search, view details and contact owners using their displayed telephone or WhatsApp number. Students with an account can save listings in their browser. Owners can register, sign in and manage their own listings. Online payments, room reservations, property verification and image file uploads are outside the scope of this mini project.

2. Development Technologies

Layer	Technology	Reason
Frontend	React, Vite, CSS	Component-based UI and responsive design
Backend	Node.js, Express	REST API and request handling
Database	MongoDB, Mongoose	Persistent records and data models
Authentication	JWT, bcryptjs	Signed sessions and password hashes
Other	React Router, Lucide	Navigation and lightweight iconography

3. System Design

3.1 Architecture

The React frontend sends HTTP requests to the Express API. Express handles authentication, validates incoming data and uses Mongoose to read or write MongoDB documents. Successful API responses are displayed in the frontend. The student-side favourite list is stored separately in localStorage.

React Client → Express REST API → Mongoose → MongoDB

3.2 Database Design and ERD

A User document contains account information and a role (student or owner). A Boarding document contains the property information and an owner reference. One owner can have multiple boarding listings. The owner field is a MongoDB ObjectId referencing the User collection.

USER (1)	BOARDING (Many)
_id – Primary Key	_id – Primary Key
name, email (unique)	owner – ObjectId → User._id
password – bcrypt hash	title, description, location, city
role – student | owner	price, deposit, room type, gender
createdAt, updatedAt	beds, baths, availableSpots
	amenities[], image, phone
	createdAt, updatedAt

3.3 Key REST Endpoints

Method and URL	Operation	Permission
GET /api/boardings	List / search / filter / sort	Public
GET /api/boardings/:id	View one boarding	Public
POST /api/boardings	Create boarding	Owner
PUT /api/boardings/:id	Update boarding	Listing owner
DELETE /api/boardings/:id	Delete boarding	Listing owner
POST /api/auth/login	User login	Public

4. Features and User Interface

4.1 Student Pages

The homepage contains a search form and featured boarding cards. The Explore page supports search by keyword or area, filtering by room type, preferred tenants and maximum rent, plus price sorting. Each boarding has a detail page with amenities, availability, deposit, owner name and contact links.

[INSERT YOUR ACTUAL SCREENSHOT: Homepage with hero and featured property cards]
Open the running application, take a screenshot, and replace this placeholder.

[INSERT YOUR ACTUAL SCREENSHOT: Explore page showing search, filter and sort]
Open the running application, take a screenshot, and replace this placeholder.

4.2 Property Detail and Saved Listings

A student can review key property details and use phone or WhatsApp links to contact the owner. Saved properties are stored in the browser using localStorage. This convenience feature does not replace the required MongoDB CRUD operations.

[INSERT YOUR ACTUAL SCREENSHOT: Property detail page or saved listings page]
Open the running application, take a screenshot, and replace this placeholder.

5. Full CRUD Implementation

The Boarding entity is the main database entity for this project. All four operations are performed through the REST API with MongoDB persistence.

Operation	User action / API behaviour
CREATE	Owner completes Add Boarding form; POST /api/boardings saves a new MongoDB record.
READ	Explore and detail pages read saved records; GET endpoints support listing and filtering.
UPDATE	Owner opens Edit; PUT /api/boardings/:id changes that existing MongoDB record.
DELETE	Owner confirms deletion; DELETE /api/boardings/:id removes the record.

[INSERT YOUR ACTUAL SCREENSHOT: Owner dashboard with boarding listings]
Open the running application, take a screenshot, and replace this placeholder.

[INSERT YOUR ACTUAL SCREENSHOT: Add / edit boarding form]
Open the running application, take a screenshot, and replace this placeholder.

[INSERT YOUR ACTUAL SCREENSHOT: Delete confirmation dialog]
Open the running application, take a screenshot, and replace this placeholder.

6. Validation, Error Handling and Security

The React forms use required fields, minimum lengths and numeric input limits. The Express API performs separate checks on prices, phone number format, room type, tenant preference and mandatory fields. Error messages are returned in JSON and presented on screen. JWT authentication protects owner routes. The API also compares the owner ID of each boarding before allowing edits or deletion.

7. Testing and Verification

Complete this table after running the application. Record only results you actually observed. A status is intentionally not filled in because a real MongoDB test and browser run must be done on your machine.

Test case	Expected result	Actual result / status
Add a valid property	New record appears after reload	[Fill after testing]
Search Matara	Matching Matara listings shown	[Fill after testing]
Edit monthly rent	New price remains after reload	[Fill after testing]
Delete a test property	Property no longer appears	[Fill after testing]
Submit invalid price	Error shown; invalid record not saved	[Fill after testing]
Open on mobile viewport	Navigation and cards fit screen	[Fill after testing]
Student attempts owner route	Request blocked by access control	[Fill after testing]

8. Limitations and Future Improvements

- Sample images are loaded by URL; file uploading could be added later.
- Favourite listings are stored per browser instead of per user account.
- An online booking workflow and property verification are not implemented.
- Future versions could include map integration, messaging and booking requests.

9. Conclusion

BoardEase provides a practical MERN-stack approach to student boarding discovery and owner property management. Its central database-backed CRUD workflow demonstrates forms, REST APIs, Mongoose models, routing, authorization and responsive frontend design.

References

1. MongoDB Documentation (mongodb.com/docs)
2. React Documentation (react.dev)
3. Express Documentation (expressjs.com)
4. Node.js Documentation (nodejs.org/docs)

ACTION REQUIRED Before submitting: add your actual screenshots and test results, check the final page count (10 pages maximum), and use your index number in the file name.